When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

Design/Methodology/Approach

The study follows design science research. The audit produced a taxonomy of seven failure modes. Each was matched to a design principle grounded in established theory: integrity models, fail-safe defaults, complete mediation, separation of duties, tamper-evident logging and the critique of ritualised assurance. The principles were instantiated in a new system, ERMA-GRC. Both systems were then run against identical stored data in 23 scenarios, whose protocol was time-stamped in the project repository before execution. Eighteen scenarios derive from the audited failures, four test the principles beyond them, and one was expected to fail. Every deviation from the protocol is reported.

Findings

The original system prevented the failure in 4 of the 18 audit-derived scenarios; ERMA-GRC prevented it in all 18. ERMA-GRC accepted no unauthenticated writes, rejected self-approval, recorded audit entries only under the signed-in identity, detected an edited audit entry and returned errors instead of false successes. It also prevented 3 of the 4 failures in the scenarios beyond the audit, including a race between two simultaneous approval decisions, and every stored-state outcome was read from the database stand-in, not from what either system reported. Two failures of the new design are reported with equal weight. First, a key risk indicator's status was computed correctly in the interface but could be declared by a client calling the server directly. Second, an insider holding the audit key could rewrite the evidence trail undetected after a restart. The principles also depend on one another: separation of duties relies on the integrity of the audit trail, and a rule enforced only in the client offers no assurance.

Originality/Value

The study treats assurance integrity as a design property of GRC information systems rather than an organisational outcome. It contributes three things: a failure taxonomy derived from a

real system, seven design principles with two testable propositions toward a nascent design theory, and a reusable scenario protocol. Auditors and buyers can use the principles as a review checklist. The documented boundaries point to future work: externally anchored audit trails, server-side computation of every assurance metric, and evaluation with practitioners.

Keywords: assurance integrity; governance, risk and compliance; design science research; enterprise risk management; audit trail; separation of duties

1. Introduction

Organisations increasingly record their risk management in governance, risk and compliance (GRC) information systems. A risk register, a heatmap, an approved assessment and an audit trail are no longer working notes. Boards, external auditors and regulators rely on them as evidence that risks are understood and controlled (Power, 1997). The value of such a system therefore depends on whether what it shows is true.

This study began with a code and runtime audit of an existing GRC application, developed before this study and documented as production-ready. The audit found that the software produced assurance evidence with no underlying record. It invented ratings for unrated risks from record identifiers or default loss values, reset every approval to draft on the next save, kept an "immutable" audit trail in the browser where anyone could rewrite it, and reported failed database writes as successes. None of these were exotic security flaws. They were ordinary design decisions, and each turned the system into a source of risk rather than a control over it.

Within the literature reached by our search, this problem has not been addressed. GRC information systems research has defined the field and studied adoption and value (Papazafeiropoulou & Spanaki, 2016; Racz et al., 2010). Research on enterprise risk management warns that assurance can decay into ritual (Power, 2009). The relevant mechanisms are well understood in generic form (Clark & Wilson, 1987; Saltzer & Schroeder, 1975). We found no study that connects application design failures to corrupted assurance evidence, or that tests design principles against a real system.

We therefore ask which design failures in a GRC information system compromise the integrity of its assurance evidence (RQ1); which design principles, grounded in established kernel theories, address these failures (RQ2); and to what extent an instantiation of these principles prevents the failures, compared with the original system (RQ3). Using design science research, we contribute a failure taxonomy, seven design principles and a time-stamped scenario evaluation of the artifact, ERMA-GRC. Sections 2 to 6 present the literature, method, results, implications, and limitations with future research.

2. Literature Review

2.1 Conceptual Background

Enterprise risk management (ERM) is the organisation-wide identification, assessment, treatment and monitoring of risk. *Governance, risk and compliance (GRC) information systems* integrate these activities and keep their records (Racz et al., 2010). *Assurance* is the confidence those records give boards, auditors and regulators that risks are known and controlled. We define *assurance integrity* as the property that every assurance item a GRC system presents is either a recorded, attributable action or an explicit statement that no such record exists. Such items include a rating, an approval, an exposure figure and an audit entry. Assurance integrity is therefore a property of the system's design, not of the risks it manages.

2.2 Review of Previous Studies

Organisational research shows that ERM is enacted rather than simply adopted. Arena et al. (2010) trace how risk rationalities, experts and technologies combine differently across firms; Mikes (2009) identifies distinct "calculative cultures"; Mikes and Kaplan (2015) argue that a single ERM template is itself a risk; and Meidell and Kaarbøe (2017) show that the tools a risk function builds shape its influence on decisions. Power (1997) warns that verification rituals can produce auditability rather than assurance, and Power (2009) calls ERM at its weakest a "risk management of nothing". Technology is therefore part of how risk is produced, and its design is a legitimate object of study.

Research on GRC information systems has defined the field (Racz et al., 2010). It has identified implementation characteristics and success factors from expert interviews (Papazafeiropoulou & Spanaki, 2016), and described an adoption lifecycle and value drivers (Spanaki & Papazafeiropoulou, 2016). It asks how these systems are adopted and what value they deliver, not whether their evidence can be trusted.

The mechanisms the artifact relies on are established in generic form: well-formed transactions, integrity verification and separation of duty (Clark & Wilson, 1987); fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975); separation of duties in workflows (Botha & Eloff, 2001); and tamper-evident logs, whether hash chains with evolving keys (Schneier & Kelsey, 1999) or history trees whose integrity depends on external auditors holding commitments (Crosby & Wallach, 2009). Continuous assurance monitors business-process controls through analytics (Alles et al., 2006; Vasarhelyi et al., 2004); its focus is the monitored processes, not design faults in the monitoring software. Table 1 summarises the seven closest studies. Five of them were assessed from their abstracts only.

Table 1. *Summary of selected literature*

Author(s)	Context	Methodology	Key Findings	Gap Identified
Racz et al. (2010)	Integrated GRC	Literature analysis, definition	A frame of reference for GRC research	Not its focus: artifact design and evaluation
Papazafeiropoulou & Spanaki (2016)	GRC IS implementation	Literature review and expert interviews	Implementation characteristics and success factors	Integrity of the evidence the systems produce
Spanaki & Papazafeiropoulou (2016)	GRC IS adoption	Qualitative study	Adoption lifecycle; integration, optimisation and information as value drivers	Not its focus: trustworthiness of that information
Arena et al. (2010)	ERM in organisations	Comparative case study	ERM varies with its rationalities, experts and technologies	Not its focus: how a technology's design distorts risk information
Alles et al. (2006)	Continuous control monitoring	Pilot implementation (Siemens)	Business-process controls can be monitored continuously	Not its focus: design faults in the monitoring software

Author(s)	Context	Methodology	Key Findings	Gap Identified
Botha & Eloff (2001)	Workflow access control	Conceptual model	Static and dynamic separation-of-duties enforcement	Not its focus: approval semantics of risk assessments
Crosby & Wallach (2009)	Secure logging	Data structure and analysis	Efficient tamper-evident logs, provided auditors hold commitments	Business workflows and other failure types

2.3 Research Gap

Our search covered web-indexed sources, with every DOI verified. Within the literature it reached, no study examines how the design of a GRC application can itself corrupt the assurance evidence it presents: by fabricating values, overwriting decisions, accepting unauthenticated authority or reporting failures as success. GRC information systems research stops at adoption and value. Mechanism research does not address the evidence that risk and control self-assessment (RCSA) produces: ratings, approvals, appetite exposure and indicator status. The gap is therefore not a missing mechanism. It is missing design knowledge that maps application failures to corrupted evidence, and that tests principles against a real system.

2.4 Research Framework

The study links failure modes observed in a real system (RQ1), design principles grounded in the kernel theories above (RQ2), the mechanisms that instantiate them, and scenario outcomes that test whether the failures are prevented (RQ3). Together these contribute components of a design theory (Gregor & Jones, 2007): purpose and scope, constructs, principles of form and function, and testable propositions (Section 5). Each principle pairs an aim with a mechanism and a kernel-theory rationale (Gregor et al., 2020), as stated in Table 2.

3. Methodology

3.1 Research Design

We followed design science research (DSR; Hevner et al., 2004), entering the design science research methodology (DSRM; Peffers et al., 2007) at problem identification. Its six activities were: (1) problem identification through the audit, yielding seven failure modes (RQ1); (2) objectives, preventing each failure mode; (3) design of seven principles and their instantiation, ERMA-GRC, a web application whose server mediates every read and write (RQ2); (4) demonstration of the risk and control self-assessment (RCSA) workflow on Oracle Autonomous Database; (5) evaluation (RQ3); and (6) communication, this paper. The evaluation is one artificial, summative episode of the Technical Risk and Efficacy strategy of the Framework for Evaluation in Design Science (FEDS; Venable et al., 2016); naturalistic episodes remain.

3.2 Data Collection

Evidence came from the audit and a scenario experiment whose protocol was time-stamped in the project repository, not a public registry, before execution. The original system (the baseline) and ERMA-GRC ran against one in-memory database stand-in seeded with identical rows (Figure 1). Each scenario had one recorded run per system, keeping its HTTP status, stored rows, screen text and screenshot.

3.3 Measures and Instruments

The unit of analysis is a scenario run. Eighteen scenarios operationalise the failure modes, four test the principles beyond the audited defects, and one boundary scenario was expected to fail. Each run was coded PREVENTED, OCCURRED or not applicable against a criterion registered in advance. Stored-state outcomes were read from the stand-in, and display outcomes from screen text.

3.4 Data Analysis

We count PREVENTED scenarios per failure mode, without inferential statistics, because outcomes are deterministic and the scenarios form a test suite, not a sample.

All deviations are reported in a replication package available on request. The test scripts were piloted on a separate copy after registration. Choices where the protocol was silent, such as an approver as editor in S2.2, were fixed before each run. The rule coding G4 by its stricter path was not registered, and one run (S5.1) was repeated after a script fault.

Use of generative AI. Claude (Claude Opus 5.5, Anthropic) was used under the authors' direction to assist the audit, implement much of ERMA-GRC, draft the scenario protocol, write and run the test scripts, propose outcome codings, produce the figures, support the literature search and DOI checks, and draft text.

The authors set the questions, approved each stage including the protocol before execution, verified sources and results, revised the text and take full responsibility. No AI tool is an author.

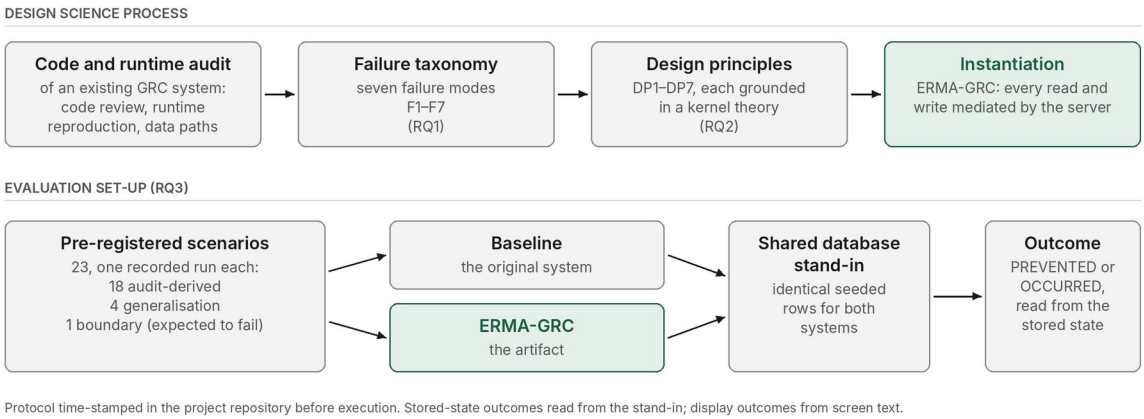


Figure 1. DSR process and evaluation set-up. The audit of the baseline yields failure modes F1–F7, which inform principles DP1–DP7, which are instantiated in ERMA-GRC. Both systems run the 23 scenarios against one shared database stand-in.

4. Results and Discussion

4.1 Results

RQ1 and RQ2. Table 2 maps each failure mode to the principle that counters it, that principle's kernel theory, and its mechanism in ERMA-GRC.

Table 2. *Failure modes, design principles, kernel theories and mechanisms*

Failure mode (RQ1)	**Design principle (RQ2)**	**Kernel theory**	**Mechanism in ERMA-GRC**
F1 Fabricated assurance values	DP1 Evidence provenance	Clark & Wilson (1987); Power (1997)	No derived ratings; unrated risks shown as "Not rated" and kept off the heatmap
F2 Decisions overwritten by defaults	DP2 Decision preservation	Clark & Wilson (1987); Saltzer & Schroeder (1975)	Defaults fill only missing fields at creation; approvals change only through authorised transitions
F3 Client-side or absent enforcement	DP3 Server-enforced, identity-bound separation of duties	Saltzer & Schroeder (1975); Botha & Eloff (2001)	Signed server session; deny-by-default permissions; maker ≠ checker; a conflicting decision is refused; deactivation revokes sessions
F4 Mutable, forgeable evidence trail	DP4 Tamper-evident, actor-bound trail	Schneier & Kelsey (1999); Crosby & Wallach (2009)	Keyed hash chain with a static key; actor taken from the session; verification shown to users
F5 Silent failure reported as success	DP5 Fail-visible operation	Saltzer & Schroeder (1975)	Database failures returned as error statuses; no fallback store
F6 Fragmented system of record	DP6 Single system of record	Clark & Wilson (1987)	Risk and module records in the database; accounts and audit chain on the API server, the chain copied to the database
F7 Destructive actions on fabricated records	DP1 Evidence provenance	as for F1	Empty states instead of placeholder rows
F1 (metrics)	DP7 Computed, not declared, metrics	Vasarhelyi et al. (2004); Cox (2008)	Indicator status derived from thresholds and direction, in the client

RQ3. On the 18 audit-derived scenarios, ERMA-GRC prevented the failure in all 18, and the baseline in 4 (Table 3). The baseline stored DRAFT after an approver set APPROVED, accepted writes with no session or with only a self-declared administrator header, recorded an

audit entry under a claimed "CEO" identity, reported success while the database was unreachable, and invented two scored risks, each with a delete button, for a cycle that had none. ERMA-GRC prevented three of the four generalisation failures. Of two simultaneous approval decisions it kept one and refused the other; it rejected a write from a deactivated user's still-valid session; and it rejected a path-traversal identifier before any database request. The baseline's 0/4 here largely repeats its F2 and F3 defects, so it is not independent evidence. Two variants are reported for balance. When an editor without approval rights changed an approved risk (S2.2 variant), ERMA-GRC returned it to review by design, which the literal S2.2 criterion codes as OCCURRED. The same simultaneous-decision race on a record's status (G1 variant) was prevented.

Table 3. *Scenario outcomes (PREVENTED / runnable)*

Scenarios	**Baseline**	**ERMA-GRC**
F1 Fabricated values (S1.1–S1.4)	3 / 4	4 / 4
F2 Decisions overwritten (S2.1–S2.2)	0 / 2	2 / 2
F3 Enforcement (S3.1–S3.4)	0 / 4	4 / 4
F4 Evidence trail (S4.1–S4.3)	1 / 3	3 / 3
F5 Silent failure (S5.1–S5.3)	0 / 3	3 / 3
F6 Fragmented record (S6.1)	0 / 1	1 / 1
F7 Fabricated-row deletion (S7.1)	0 / 1	1 / 1
Generalisation (G1–G4)	0 / 4	3 / 4
Boundary (L1, expected to fail)	0 / 1	0 / 1
Total	**4 / 23**	**21 / 23**

Where the baseline passed. The baseline met four criteria. It displayed a stored statement and single loss expectancy (SLE) faithfully (S1.2). It did not show a missing approval status as Approved, displaying "Draft" as ERMA-GRC also does (S1.3). Its fresh audit trail held no invented entries (S1.4). It attributed an audit entry to the acting persona, though that identity was self-asserted (S4.3). S1.3 and S1.4 contradict the audit report on the screens examined; for S1.4, the code that seeds invented audit entries exists but never runs on a fresh installation.

Where ERMA-GRC failed. ERMA-GRC failed twice. In G4, a key risk indicator (KRI) reading received its status by rule in the interface, but the API stored a status declared by the client; the baseline behaves the same way. In L1, an insider holding the audit key re-chained an edited log offline, and verification passed after a restart.

Figure 2 shows two of the baseline's failures next to ERMA-GRC's handling of the same stored data.

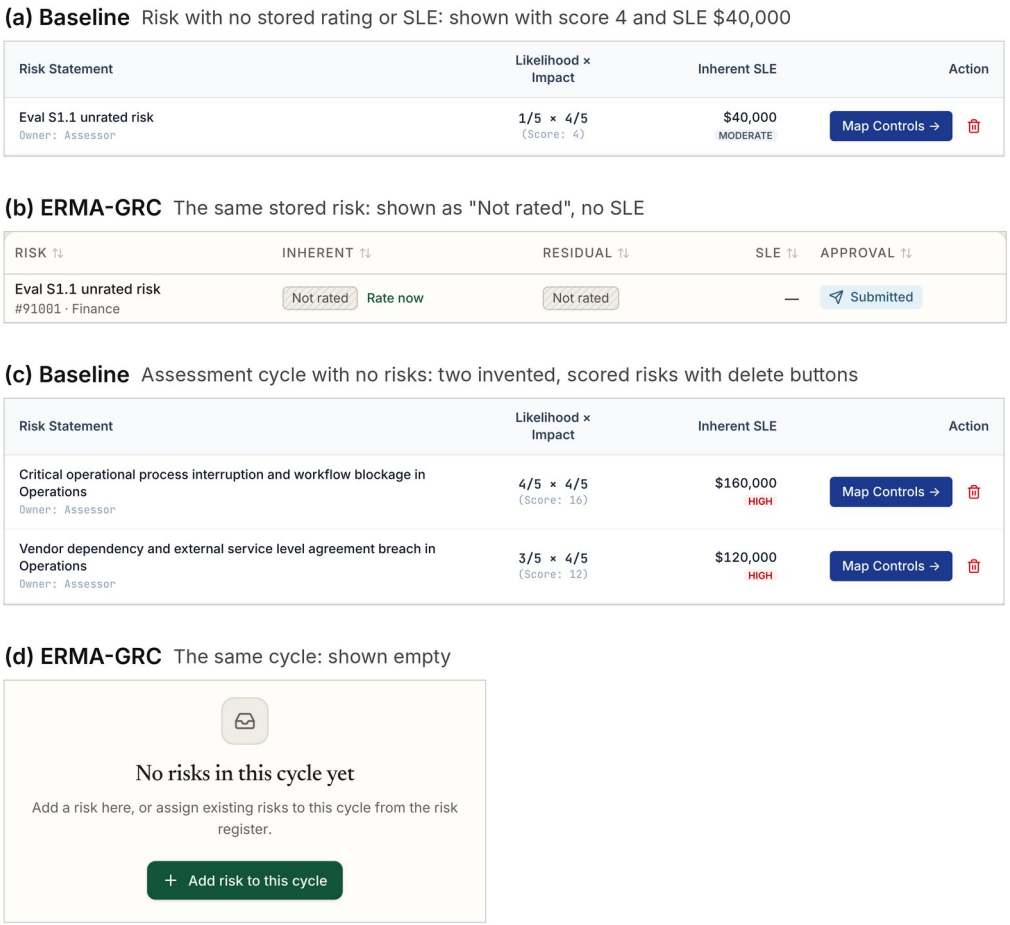


Figure 2. The same stored data in both systems. (a) The baseline shows a risk with no stored rating or SLE with a score of 4 and an SLE of $40,000 (scenario S1.1). (b) ERMA-GRC shows the same risk as "Not rated". (c) For an assessment cycle that has no risks, the baseline shows two invented, scored risks, each with a delete button (S7.1). (d) ERMA-GRC shows the cycle as empty. These are illustrative re-captures, taken after the evaluation from the renamed build with the same set-up; unrelated columns are hidden.

4.2 Discussion

The failures are assurance failures. The baseline produced scores, approvals and audit entries that were disconnected from its stored state: a software-made "risk management of nothing" (Power, 2009). The mechanisms that counter this are established: fail-safe defaults and complete mediation (Saltzer & Schroeder, 1975), separation of duties (Botha & Eloff, 2001), and well-formed transactions with integrity verification (Clark & Wilson, 1987). DP1, DP2 and DP7 specialise these integrity ideas to assurance evidence: they ask whether a displayed rating, decision or metric is evidence at all.

The principles depend on each other. ERMA-GRC reads a record's creator, and hence who may not approve it, from its audit trail. In L1, rewriting one creation entry changed who was barred from approving that record. DP3 therefore rests on DP4. DP4 in turn rests on keys held outside the application. Schneier and Kelsey (1999) evolve their keys; ERMA-GRC's key is static. Without external commitments, whoever holds that key can rewrite history (Crosby & Wallach, 2009).

Enforcement must sit at the server. G4 shows that a rule checked only in the client is a convention, not a control. Integrity remains necessary but not sufficient: faithful records of a poorly calibrated risk matrix are still poor risk information (Cox, 2008).

5. Implications

Theoretical implications. The study recasts assurance integrity as a design property of GRC systems. It contributes components of a nascent design theory (Gregor & Jones, 2007): purpose and scope, constructs, seven principles, and two testable propositions. P1: in a GRC system, separation of duties is only as strong as the integrity of the evidence trail it reads. P2: an assurance rule enforced only in the client can be defeated by any authenticated client.

Practical implications. Buyers and auditors can turn the principles into five tests to run before purchase: whether an unrated risk is shown as unrated; whether an ordinary save can overwrite an approval; whether authority comes from a verified session; whether the audit trail can be verified and is anchored outside the vendor's control; and whether a failed write can ever appear as a success.

Societal implications. Regulators and the public increasingly rely on compliance claims produced by software. Systems that cannot fabricate evidence make those claims more trustworthy, and they make accountability harder to evade.

6. Conclusion, Limitations and Future Research

This study asked how a GRC system's design can corrupt the assurance evidence it produces, and how to prevent it. An audit yielded seven failure modes, and seven principles grounded in established theory address them. ERMA-GRC, which instantiates the principles, prevented every audit-derived failure; the original system prevented four of eighteen. ERMA-GRC's two failures are part of the contribution: a metric enforced only in the client, and an audit trail that its key holder can rewrite.

Limitations. The evidence comes from one case. The authors built the artifact and wrote the scenarios with an AI assistant that took part in every stage, from audit to outcome coding; time-stamping the protocol and reporting every outcome reduce this bias but do not remove it. The evaluation was artificial. Both systems ran against a stand-in derived from ERMA-GRC's own test double, so the concurrency and outage results depend on how faithfully it imitates the database service, and the display scenarios contained no rated risk as a positive control. The literature search lacked Scopus and Web of Science, and no commercial product was compared.

Future research. Next steps are naturalistic evaluation with risk and audit professionals, multiple-case studies across GRC products, externally anchored audit commitments, server-side computation of every assurance metric, and longitudinal study of whether integrity-preserving systems change how organisations use risk information.

References

Alles, M., Brennan, G., Kogan, A., & Vasarhelyi, M. A. (2006). Continuous monitoring of business process controls: A pilot implementation of a continuous auditing system at Siemens. *International Journal of Accounting Information Systems, 7*(2), 137–161. https://doi.org/10.1016/j.accinf.2005.10.004

Arena, M., Arnaboldi, M., & Azzone, G. (2010). The organizational dynamics of enterprise risk management. *Accounting, Organizations and Society, 35*(7), 659–675. https://doi.org/10.1016/j.aos.2010.07.003

Botha, R. A., & Eloff, J. H. P. (2001). Separation of duties for access control enforcement in workflow environments. *IBM Systems Journal, 40*(3), 666–682. https://doi.org/10.1147/sj.403.0666

Clark, D. D., & Wilson, D. R. (1987). A comparison of commercial and military computer security policies. In *Proceedings of the 1987 IEEE Symposium on Security and Privacy* (pp. 184–194). IEEE. https://doi.org/10.1109/SP.1987.10001

Cox, L. A., Jr. (2008). What's wrong with risk matrices? *Risk Analysis, 28*(2), 497–512. https://doi.org/10.1111/j.1539-6924.2008.01030.x

Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium* (pp. 317–334). USENIX Association. https://www.usenix.org/legacy/event/sec09/tech/full_papers/crosby.pdf

Gregor, S., Chandra Kruse, L., & Seidel, S. (2020). Research perspectives: The anatomy of a design principle. *Journal of the Association for Information Systems, 21*(6), 1622–1652. https://doi.org/10.17705/1jais.00649

Gregor, S., & Jones, D. (2007). The anatomy of a design theory. *Journal of the Association for Information Systems, 8*(5), 312–335. https://doi.org/10.17705/1jais.00129

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–106. https://doi.org/10.2307/25148625

Meidell, A., & Kaarbøe, K. (2017). How the enterprise risk management function influences decision-making in the organization – A field study of a large, global oil and gas company. *The British Accounting Review, 49*(1), 39–55. https://doi.org/10.1016/j.bar.2016.10.005

Mikes, A. (2009). Risk management and calculative cultures. *Management Accounting Research, 20*(1), 18–40. https://doi.org/10.1016/j.mar.2008.10.005

Mikes, A., & Kaplan, R. S. (2015). When one size doesn't fit all: Evolving directions in the research and practice of enterprise risk management. *Journal of Applied Corporate Finance, 27*(1), 37–40. https://doi.org/10.1111/jacf.12102

Papazafeiropoulou, A., & Spanaki, K. (2016). Understanding governance, risk and compliance information systems (GRC IS): The experts view. *Information Systems Frontiers, 18*(6), 1251–1263. https://doi.org/10.1007/s10796-015-9572-3

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Power, M. (1997). *The audit society: Rituals of verification*. Oxford University Press.

Power, M. (2009). The risk management of nothing. *Accounting, Organizations and Society, 34*(6–7), 849–855. https://doi.org/10.1016/j.aos.2009.06.001

Racz, N., Weippl, E., & Seufert, A. (2010). A frame of reference for research of integrated governance, risk and compliance (GRC). In B. De Decker & I. Schaumüller-Bichl (Eds.), *Communications and multimedia security* (Lecture Notes in Computer Science, Vol. 6109, pp. 106–117). Springer. https://doi.org/10.1007/978-3-642-13241-4_11

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Schneier, B., & Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security, 2*(2), 159–176. https://doi.org/10.1145/317087.317089

Spanaki, K., & Papazafeiropoulou, A. (2016). The implementation of governance, risk, and compliance IS: Adoption lifecycle and enterprise value. *Information Systems Management, 33*(4), 302–315. https://doi.org/10.1080/10580530.2016.1220214

Vasarhelyi, M. A., Alles, M. G., & Kogan, A. (2004). Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting, 1*(1), 1–21. https://doi.org/10.2308/jeta.2004.1.1.1

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems, 25*(1), 77–89. https://doi.org/10.1057/ejis.2014.36